Data Engineering

Study Notes

UNIT 1

Fundamentals of Data Engineering and ETL

Rahul Inchal

1. Introduction to Data Engineering

Data Engineering is the branch of engineering that focuses on designing, building and maintaining the systems and infrastructure that allow organizations to collect, store, process and deliver data reliably and at scale. A Data Engineer builds the "pipes" through which raw data flows and turns it into a clean, usable form for analysts, data scientists and business applications.

Why Data Engineering Matters

- Organizations generate huge volumes of data every day from apps, sensors, transactions and logs.
- Raw data is messy, scattered and unusable in its original form.
- Data engineers build pipelines that make this data reliable, accessible and analysis-ready.
- Good data engineering is the foundation for Business Intelligence (BI), Machine Learning (ML) and Data Science.

Key Responsibilities of a Data Engineer

1. Designing and building data pipelines (ETL/ELT).
2. Managing databases, data warehouses and data lakes.
3. Ensuring data quality, consistency and security.
4. Optimizing pipelines for performance and scalability.
5. Collaborating with data analysts, data scientists and software engineers.

> Note: Data Engineering sits at the intersection of Software Engineering, Database Systems and Distributed Computing.

2. Data Lifecycle and Data Pipelines

The Data Lifecycle

The data lifecycle describes every stage data passes through — from where it is created to how it is finally used. Understanding this lifecycle helps a data engineer decide where to add validation, storage and transformation steps.

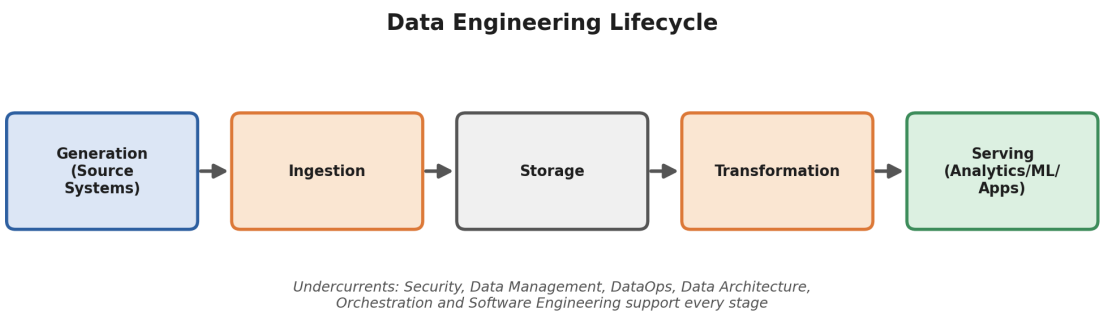


Figure 2.1: The Data Engineering Lifecycle

- Generation: Data is created at the source (applications, sensors, transactions, user activity).
- Ingestion: Data is collected and brought into the system (batch or streaming).
- Storage: Data is stored in databases, data lakes or data warehouses.
- Transformation: Data is cleaned, validated, and reshaped for use.
- Serving: Processed data is delivered to dashboards, ML models, and applications.

What is a Data Pipeline?

A data pipeline is a series of automated steps that move data from one or more sources to a destination, applying processing along the way. Pipelines can run on a schedule (batch) or continuously (streaming).

Source: Where data originates — databases, APIs, files, IoT sensors, logs.

Pipeline stages: Extraction → Processing/Transformation → Loading/Delivery.

Destination: Where processed data lands — a data warehouse, data lake, or downstream application.

3. Introduction to ETL: Extract, Transform and Load

ETL is the most common pattern used to move and prepare data for analytics. It stands for Extract, Transform and Load — three sequential stages that convert raw, scattered data into a clean, structured form ready for reporting and analysis.

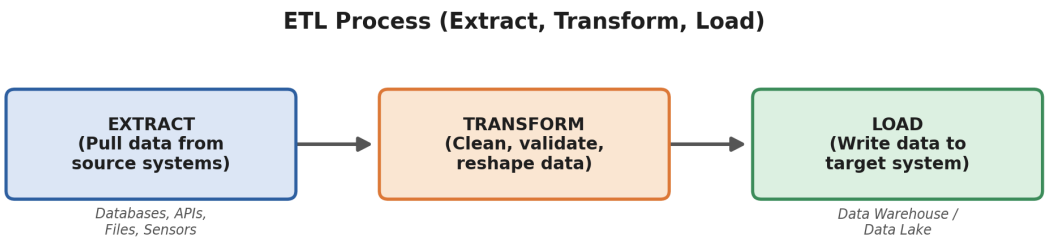


Figure 3.1: The three stages of ETL

The Three Stages

Extract: Data is pulled/read from one or more source systems such as relational databases, APIs, flat files, or sensors. This stage focuses on retrieving raw data without altering it.

Transform: The extracted data is cleaned, validated, standardized, aggregated and reshaped into the format required by the target system. This is usually the most complex stage.

Load: The transformed data is written into the target system — typically a data warehouse or data mart — where it becomes available for querying and reporting.

ETL vs ELT (Quick Comparison)

Aspect	ETL (Extract-Transform-Load)	ELT (Extract-Load-Transform)
Transformation location	Before loading (staging server)	After loading (inside target DW)
Best suited for	Structured, smaller datasets	Big Data, cloud data warehouses
Speed of loading	Slower (transform first)	Faster (load raw data first)
Example tools	Informatica, SSIS, Talend	Snowflake, BigQuery, Databricks

4. ETL Architecture and Workflow

A typical ETL architecture involves multiple systems working together — source systems, a staging area, transformation logic, the target data warehouse, and an orchestration layer that schedules and monitors the entire workflow.

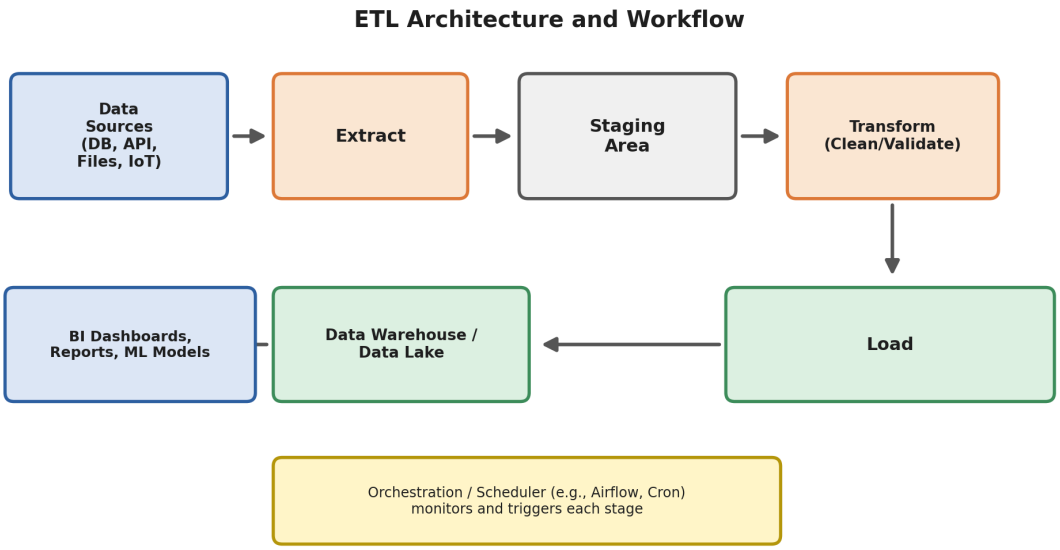


Figure 4.1: End-to-end ETL architecture

Components of ETL Architecture

- Data Sources: Operational databases, third-party APIs, flat files, IoT devices.
- Staging Area: A temporary storage layer where raw extracted data is held before transformation — isolates source systems from transformation logic.
- Transformation Engine: Applies cleaning, business rules, joins, and aggregations.
- Target/Data Warehouse: Stores the final, query-ready data.
- Orchestration/Scheduler: Tools like Apache Airflow, Cron, or Control-M trigger and monitor each stage, handle retries and failures.

Typical ETL Workflow (Step by Step)

1. Identify and connect to data sources.
2. Extract data into a staging area.
3. Apply data cleaning and transformation rules.
4. Validate transformed data against quality rules.
5. Load data into the data warehouse/data mart.
6. Log the job status and notify on success/failure.

5. Types of Data: Structured, Semi-Structured and Unstructured

Data engineers must design pipelines that can handle very different kinds of data. Understanding these categories helps in choosing the right storage system and processing approach.

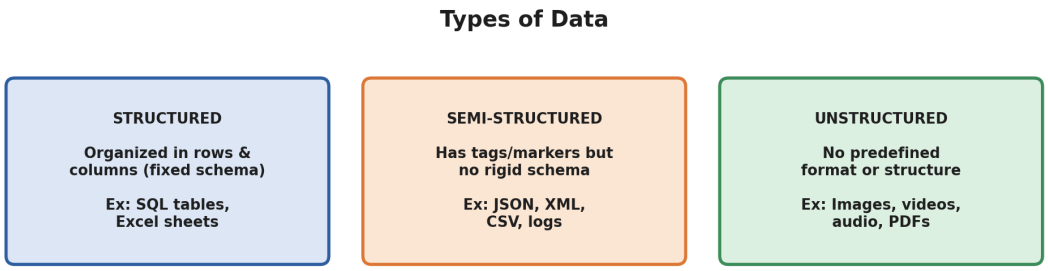


Figure 5.1: Three broad categories of data

Type	Description	Examples
Structured	Highly organized, follows a fixed schema (rows and columns)	RDBMS tables, Excel sheets, CSV with fixed columns
Semi-Structured	Has some organizational tags/markers but no rigid schema	JSON, XML, YAML, log files, emails
Unstructured	No predefined format or organization	Images, audio, video, PDFs, social media posts

6. Data Sources and Data Destinations

Common Data Sources

- Relational Databases (MySQL, PostgreSQL, Oracle)
- NoSQL Databases (MongoDB, Cassandra)
- APIs and Web Services (REST, GraphQL)
- Flat Files (CSV, JSON, XML, Parquet, Avro)
- Streaming Sources (IoT sensors, application logs, clickstreams, message queues like Kafka)

Common Data Destinations

- Data Warehouses (Snowflake, Amazon Redshift, Google BigQuery) — for structured, query-optimized analytics.
- Data Lakes (Amazon S3, Azure Data Lake, HDFS) — for storing raw data in any format at scale.
- Data Marts — subject-specific subsets of a data warehouse for a department/team.
- Operational systems and dashboards for real-time decision-making.

7. Data Cleaning and Data Transformation

Data Cleaning

Data cleaning is the process of detecting and correcting (or removing) inaccurate, incomplete, duplicate, or irrelevant parts of data.

- Handling missing values (drop, impute with mean/median/mode, or flag).
- Removing duplicate records.
- Correcting inconsistent formats (dates, units, casing).
- Detecting and treating outliers.
- Standardizing categorical values (e.g., "M"/"Male"/"male" → "Male").

Data Transformation

Transformation reshapes cleaned data into the structure required by the destination system or analysis.

- Normalization / Standardization of numeric values.
- Aggregation (sum, average, count by group).
- Joining data from multiple sources.
- Encoding (e.g., converting categorical data to numeric for ML).
- Filtering and deriving new calculated columns.

8. Data Quality and Validation

Data quality refers to how well data serves its intended purpose. Poor-quality data leads to wrong business decisions, so validation checks are built into pipelines to catch problems early.

Dimensions of Data Quality

Dimension	Meaning
Accuracy	Data correctly reflects the real-world value.
Completeness	No missing required fields/records.
Consistency	Same data does not conflict across systems.
Timeliness	Data is available and up-to-date when needed.
Uniqueness	No unwanted duplicate records.
Validity	Data conforms to defined formats/rules (e.g., valid email, date range).

Validation Techniques

- Schema validation (data types, required fields).
- Range and constraint checks (e.g., age between 0–120).
- Referential integrity checks (foreign keys exist).
- Automated data quality frameworks (e.g., Great Expectations, dbt tests).

9. Batch Processing and Real-Time Processing

Data pipelines can process data in two broad modes depending on how quickly results are needed.

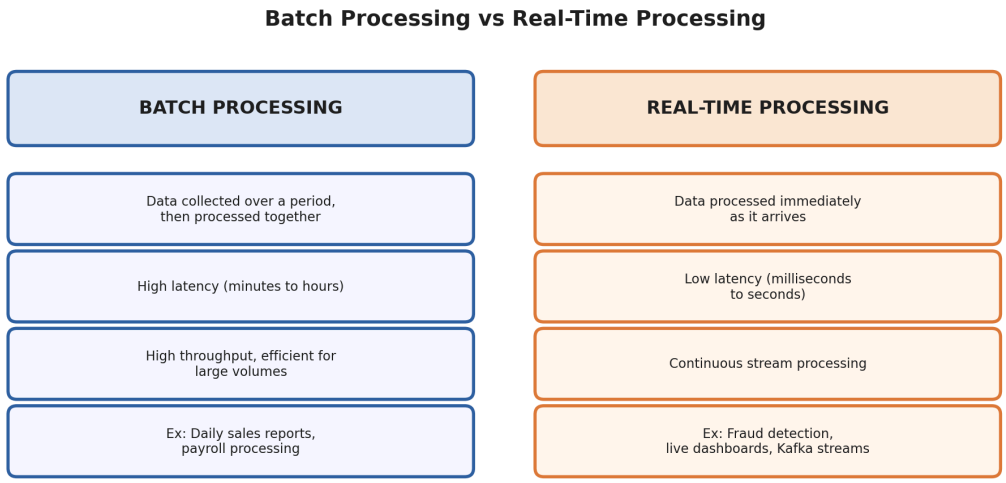


Figure 9.1: Batch processing vs Real-time (stream) processing

Aspect	Batch Processing	Real-Time (Stream) Processing
Data handling	Processed in fixed-size chunks/batches	Processed continuously, record by record
Latency	High (minutes to hours)	Low (milliseconds to seconds)
Use cases	Payroll, daily/monthly reports, ETL loads	Fraud detection, live dashboards, alerts
Common tools	Apache Sqoop, traditional ETL tools, Spark (batch)	Apache Kafka, Apache Flink, Spark Streaming
Resource usage	Efficient for large volumes, scheduled runs	Requires always-on infrastructure

10. Data Warehousing Concepts: OLTP vs OLAP, Star and Snowflake Schema

What is a Data Warehouse?

A Data Warehouse is a central repository that stores large volumes of structured, historical data, specifically designed for analysis and reporting rather than daily transactions.

To understand data warehouses properly, we first need to understand the two types of systems that work with data: OLTP and OLAP.

OLTP vs OLAP

OLTP (Online Transaction Processing)

OLTP systems are used to manage day-to-day transactional operations of a business. They handle a large number of short, simple transactions like insert, update, and delete.

Characteristics:

- Handles real-time, everyday operations
- Large number of short transactions (many users at once)
- Data is current and frequently updated
- Optimized for speed of transactions, not analysis
- Database design is highly normalized (to avoid data duplication)

Example: When you book a movie ticket on BookMyShow, the system:

- Checks seat availability (read)
- Reserves your seat (update)
- Confirms your payment (insert new transaction record)

This happens in seconds and involves thousands of users doing this simultaneously. This is OLTP in action.

Other examples: ATM withdrawal, online shopping checkout, college fee payment portal, hospital patient registration.

OLAP (Online Analytical Processing)

OLAP systems are used to analyze large volumes of historical data to support business decisions. They handle complex queries that summarize data rather than modify it.

Characteristics:

- Used for analysis, reporting, and decision-making
- Handles complex queries over huge volumes of historical data
- Data is read-only (rarely updated); refreshed periodically (e.g., nightly)

- Optimized for speed of analysis, not transactions
- Database design is often denormalized (to speed up reading/reporting)

Example: At the end of the year, BookMyShow's management wants to know: "Which movie genre sold the most tickets in Maharashtra over the last 12 months?" This requires scanning millions of past transaction records and summarizing them — this is OLAP in action, typically run on the data warehouse, not the live booking database.

Other examples: Sales trend dashboards, yearly financial reports, customer buying pattern analysis.

Comparison Table:

Feature	OLTP	OLAP
Purpose	Manage daily transactions	Analyze historical data
Users	Many (customers, staff)	Few (analysts, managers)
Data	Current, frequently updated	Historical, mostly read-only
Query type	Simple (insert/update/delete)	Complex (aggregations, trends)
Design	Normalized (reduces redundancy)	Denormalized (faster reads)
Example system	Banking app, e-commerce checkout	Sales dashboard, BI reports
Example tool	MySQL, PostgreSQL	Amazon Redshift, Google BigQuery

Simple Analogy:

- OLTP = The cash counter of a supermarket — fast, one bill at a time, constantly busy
- OLAP = The manager's office reviewing the whole month's sales records to decide what to restock

---

Star Schema and Snowflake Schema

In a data warehouse, data is organized using special table designs called schemas, which make it easy to run analytical queries. The two most common schemas are Star Schema and Snowflake Schema. Both are built around two types of tables:

- Fact Table – Stores measurable, numeric business data (e.g., sales amount, quantity sold). This table is usually very large.
- Dimension Table – Stores descriptive information related to the facts (e.g., product details, customer details, date details). These tables give context to the numbers in the fact table.

Star Schema

In a Star Schema, one central Fact table is directly connected to multiple Dimension tables. Each dimension table is simple (not further broken down), so the design looks like a star when diagrammed.

Example: A retail store's sales data warehouse:

- Fact_Sales (Fact Table): Sale_ID, Product_ID, Customer_ID, Date_ID, Quantity, Total_Amount
- Dim_Product (Dimension Table): Product_ID, Product_Name, Category, Price
- Dim_Customer (Dimension Table): Customer_ID, Customer_Name, City, Age
- Dim_Date (Dimension Table): Date_ID, Day, Month, Year

Here, Fact_Sales connects directly to Dim_Product, Dim_Customer, and Dim_Date — forming a star shape, with Fact_Sales in the center.

Why use it: Simple structure, fewer joins needed, so queries run faster. Easy for beginners to understand and query.

Snowflake Schema

A Snowflake Schema is an extension of the Star Schema, where dimension tables are further broken down (normalized) into additional related tables. This creates a structure that looks like a snowflake, with branches extending outward.

Example (continuing above):

Instead of keeping all product info in one Dim_Product table, we split it further:

- Dim_Product: Product_ID, Product_Name, Category_ID, Price
- Dim_Category: Category_ID, Category_Name, Department

Now, to get a product's category info, you need to join Dim_Product → Dim_Category — an extra step (extra "branch") compared to the star schema.

Why use it: Reduces data duplication (e.g., category name isn't repeated for every product), saves storage space. But requires more joins, so queries can be slightly slower and more complex.

Comparison Table:

Feature	Star Schema	Snowflake Schema
Structure	Simple, single-level dimensions	Multi-level, normalized dimensions
Joins needed	Fewer	More
Query speed	Faster	Slightly slower (more joins)

Storage space	More (some data repeated)	Less (data normalized)
Ease of understanding	Easier	More complex
Best for	Fast reporting, simpler datasets	Large datasets where storage matters

Simple Analogy:

- Star Schema = A wheel with one hub (fact table) and spokes going straight out to simple dimension tables
- Snowflake Schema = Same wheel, but each spoke further branches into smaller sub-spokes (dimensions broken into sub-dimensions)

---

11. Introduction to NoSQL Data Models: Document, Key-Value and Columnar Stores

What is NoSQL?

NoSQL ("Not Only SQL") refers to a category of databases that do not use the traditional table-based (rows and columns) structure of relational databases. They are designed to handle large volumes of data, flexible/changing data structures, and high-speed read/write operations — especially useful for semi-structured and unstructured data.

Why NoSQL is needed: Traditional relational databases (SQL) work great for structured data with a fixed schema, but they struggle when:

- Data doesn't fit neatly into rows and columns (e.g., social media posts with varying fields)
- The application needs to scale to handle millions of users quickly
- The data structure keeps changing frequently

NoSQL databases solve these problems by being flexible and highly scalable.

There are several types of NoSQL databases. Let's look at three common ones.

---

1. Document Store

A Document Store saves data as "documents," usually in JSON or BSON format. Each document can have a different structure (fields), unlike a rigid table, and related data can be nested within a single document.

Example: A Document Store storing student profiles:

```
{
 "student_id": "101",
 "name": "Aditi Sharma",
 "courses": ["ETL", "Kafka", "Cloud Computing"],
 "address": {
   "city": "Pune",
   "pincode": "411001"
 }
}
```

Notice how "courses" is a list and "address" is a nested object — all within a single document. Another student's document could have completely different fields (e.g., an extra "scholarship" field), and that's perfectly fine in a document store.

Popular Tools: MongoDB, CouchDB

Best used for: Content management systems, product catalogs, user profiles — anywhere data has flexible, nested structure.

---

2. Key-Value Store

A Key-Value Store is the simplest NoSQL model. Data is stored as a simple pair: a unique key and its corresponding value. The value can be anything — a string, number, JSON object, or file — but the database doesn't look inside the value; it just stores and retrieves it based on the key.

Example: An e-commerce website storing shopping cart sessions:

Key	Value
user_5001_cart	["Shoes", "Watch", "Bag"]
user_5002_cart	["Laptop"]
session_token_abc123	user_5001

When the app needs the cart for user 5001, it simply looks up the key user_5001_cart and instantly gets the value — no complex searching required. This makes key-value stores extremely fast.

Popular Tools: Redis, Amazon DynamoDB, Memcached

Best used for: Caching, session management, storing user preferences, real-time leaderboard data (e.g., in gaming apps) — anywhere simple, super-fast lookups are needed.

---

3. Columnar Store (Column-Family Store)

A Columnar Store organizes data by columns rather than rows. Instead of storing a whole record together, it groups and stores values of the same column together, which makes it very efficient for analytical queries that need to scan specific columns across huge datasets.

Example: Consider sales data for millions of transactions. In a traditional row-based table, one full row (Sale_ID, Product, Amount, Date) is stored together. In a columnar store, all "Amount" values are stored together, all "Date" values are stored together, and so on.

If you want to calculate the *total sales amount* for last month, a columnar database only needs to scan the "Amount" and "Date" columns — not the entire dataset (product names, customer info, etc.) — making the query much faster.

Visual comparison:

Row-based storage: [101, Shoes, 1200, 05-Aug] [102, Bag, 800, 06-Aug]

Column-based storage: ID: [101, 102] | Product: [Shoes, Bag] | Amount: [1200, 800] | Date: [05-Aug, 06-Aug]

Popular Tools: Apache Cassandra, HBase, Google Bigtable

Best used for: Big data analytics, time-series data, large-scale reporting systems where queries analyze specific columns over massive datasets.

Comparison Table: NoSQL Data Models

Type	Stores Data As	Example Tool	Best For
Document Store	JSON/BSON documents (flexible fields, nested data)	MongoDB	Profiles, catalogs, CMS
Key-Value Store	Simple key → value pairs	Redis, DynamoDB	Caching, sessions, fast lookups
Columnar Store	Data grouped by columns	Cassandra, HBase	Big data analytics, reporting

SQL vs NoSQL (Quick Reminder):

SQL (Relational)	NoSQL
Fixed schema (tables, rows, columns)	Flexible schema
Best for structured data	Best for structured, semi-structured, unstructured data

Example: MySQL, Oracle	Example: MongoDB, Redis, Cassandra
Scales vertically (bigger machine)	Scales horizontally (more machines)

---

12. Introduction to Workflow Orchestration: Apache Airflow and DAGs

What is Workflow Orchestration?

Workflow Orchestration is the process of automatically scheduling, coordinating, and managing a series of tasks (like ETL steps) so that they run in the correct order, at the right time, with proper error handling — without manual intervention.

Why is it needed?

Think back to our ETL pipeline (Extract → Transform → Load). In real companies, this isn't just 3 simple steps — there could be 20-30 tasks: extract from 5 different sources, run 10 transformation scripts, validate data, then load into 3 different destinations. Someone (or something) needs to:

- Run these tasks in the correct order (Transform can't start before Extract finishes)
- Run them automatically at scheduled times (e.g., every night at 1 AM)
- Handle failures (e.g., retry a task if it fails, or send an alert)
- Track which tasks succeeded/failed

Doing this manually is impractical — this is where workflow orchestration tools like Apache Airflow come in.

Apache Airflow

Apache Airflow is an open-source platform used to programmatically author, schedule, and monitor workflows (pipelines). It allows data engineers to define complex pipelines as code (in Python) and have Airflow manage their execution automatically.

Key Features:

- Workflows are written as Python code (easy to version-control and reuse)
- Provides a web-based dashboard to visually monitor pipeline runs
- Automatically retries failed tasks and sends alerts/notifications
- Allows scheduling (e.g., "run daily at 2 AM", "run every Monday")
- Shows a clear graph of task dependencies (what runs before/after what)

Example use case: A company wants to automatically:

1. Extract yesterday's sales data from a database (runs at 1:00 AM)
2. Once extraction is done, clean and transform the data (runs after step 1 completes)

3. Once transformation is done, load it into the data warehouse (runs after step 2 completes)
4. Send an email notification once loading is complete

Apache Airflow can be configured to run this exact sequence automatically, every single day, and alert the team if any step fails.

What is a DAG?

DAG stands for Directed Acyclic Graph. In Airflow, a DAG represents a complete workflow — it is a collection of all the tasks you want to run, organized in a way that shows their dependencies (i.e., which task must run before another).

Let's break down the term:

- Directed – Tasks flow in one direction (Task A → Task B means A runs before B)
- Acyclic – There are no loops; a task cannot eventually depend on itself (you can't go A → B → C → A again)
- Graph – It's visually represented as a diagram of connected boxes (tasks) and arrows (dependencies)

Example DAG (our ETL pipeline described above):

[Extract Sales Data] --> [Clean & Transform Data] --> [Load into Warehouse] --> [Send Email Notification]

This is a simple straight-line DAG. Real DAGs can be more complex, with tasks branching out and joining back:

```
          --> [Extract from MySQL] --
          /                         \
[Start]                               --> [Merge Data] --> [Transform] --> [Load] --> [Notify Team]
          \                         /
           --> [Extract from API]   --
```

Here, "Extract from MySQL" and "Extract from API" can run at the same time (in parallel) since they don't depend on each other, but both must finish before "Merge Data" can begin.

Key Terms in Airflow:

Term	Meaning
DAG	The entire workflow (collection of tasks + their order)
Task	A single unit of work within the DAG (e.g., "Extract Data")

Operator	Defines what a task actually does (e.g., PythonOperator runs Python code, BashOperator runs a shell command)
Scheduler	The Airflow component that decides when to trigger a DAG run
Dependency	The rule defining which task must complete before another starts

Why Airflow is popular for ETL and Kafka-based pipelines:

- ETL pipelines have many interdependent steps — Airflow manages this complexity cleanly
- It integrates well with Kafka, databases, cloud services, and other data tools
- Failures are automatically retried and logged, improving pipeline reliability
- The visual DAG view makes it easy to understand and debug the entire pipeline at a glance

Simple Analogy: Think of a DAG like a recipe with steps that must happen in a certain order: you must chop vegetables before cooking them, and cook them before serving. You can't serve first and cook later (no going backward — that's the "acyclic" part). Some steps, like boiling rice and chopping vegetables, can happen at the same time (parallel tasks) — just like Airflow allows parallel tasks in a DAG.

Quick Revision Summary (Additional Topics)

Topic	Key Idea
OLTP vs OLAP	OLTP = fast daily transactions; OLAP = analysis of historical data
Star Schema	Central fact table directly connected to simple dimension tables
Snowflake Schema	Star schema with dimension tables further normalized/split
Document Store	Stores flexible JSON-like documents (e.g., MongoDB)
Key-Value Store	Simple key → value pairs for fast lookup (e.g., Redis)
Columnar Store	Data stored by columns for fast analytics (e.g., Cassandra)
Workflow Orchestration	Automatically scheduling and managing pipeline tasks
Apache Airflow	Python-based tool to author, schedule, and monitor workflows
DAG	Directed Acyclic Graph — represents a workflow's tasks and their order, with no loops

---

10. Practicals

The following hands-on exercises reinforce the concepts covered in this unit using Apache Kafka, a popular distributed streaming platform used for building real-time data pipelines.

Practical 1: Install Apache Kafka on a Single Node

Objective

To install and configure Apache Kafka on a single machine (single node) for learning and development purposes.

Prerequisites

- Java Development Kit (JDK 8 or highe) installed.
- A Linux/Windows/Mac machine with at least 4 GB RAM.
- Internet connection to download Kafka binaries.

Steps

1. Verify Java installation: run "java -version" in the terminal. Install a JDK if it is not present.
2. Download the latest stable Apache Kafka release (.tgz file) from the official Apache Kafka website.
3. Extract the downloaded archive using "tar -xzf kafka_x.x-x.x.x.tgz" and move into the extracted folder.
4. Start the ZooKeeper service (required by Kafka to manage broker metadata) using the bundled start script and its configuration file.
5. In a new terminal window, start the Kafka broker/server using the bundled start script and its server configuration file.
6. Verify Kafka is running by checking the terminal logs for "started (kafka.server.KafkaServer)" with no errors.

> Note: Keep the ZooKeeper terminal and Kafka broker terminal open simultaneously — closing either one stops the service.

Practical 2: Set Up a Single-Node, Single-Broker Kafka Cluster and Perform Basic Operations

Objective

To configure a single-broker Kafka cluster and perform fundamental operations such as creating a topic and producing/consuming messages.

Steps

1. Ensure ZooKeeper and the Kafka broker (from Practical 1) are both running.
2. Create a new topic using the kafka-topics script, specifying the topic name, number of partitions, and replication factor of 1 (since there is only one broker).
3. List all available topics using the kafka-topics --list command to confirm the topic was created.
4. Open a Producer console using the kafka-console-producer script pointed at the created topic, and type messages to publish them.
5. Open a Consumer console in a separate terminal using the kafka-console-consumer script with the --from-beginning flag to read all published messages.
6. Observe that messages typed in the producer terminal instantly appear in the consumer terminal, demonstrating real-time message delivery.
7. Describe the topic using kafka-topics --describe to view partition and replica details.

Expected Outcome

Students should be able to demonstrate a working single-node Kafka cluster where messages published by a producer are successfully received by a consumer in near real-time, illustrating the publish-subscribe messaging pattern used in real-time data pipelines.

> Note: This practical directly demonstrates the concept of Real-Time Processing discussed in Section 9 — Kafka is one of the most widely used tools for building streaming ETL pipelines.